

Part 3: Proposed Methodology & Architectures

Scientific Writing Part 3 · Experimental Design · CNN · Transfer Learning
Study: src/models.py · Section 7 · Technical Report

Architectures	CNN Baseline · ResNet50 · EfficientNetB0
Techniques	Conv2D · Batch Normalization · Dropout · Compound Scaling
Date	June 09, 2026

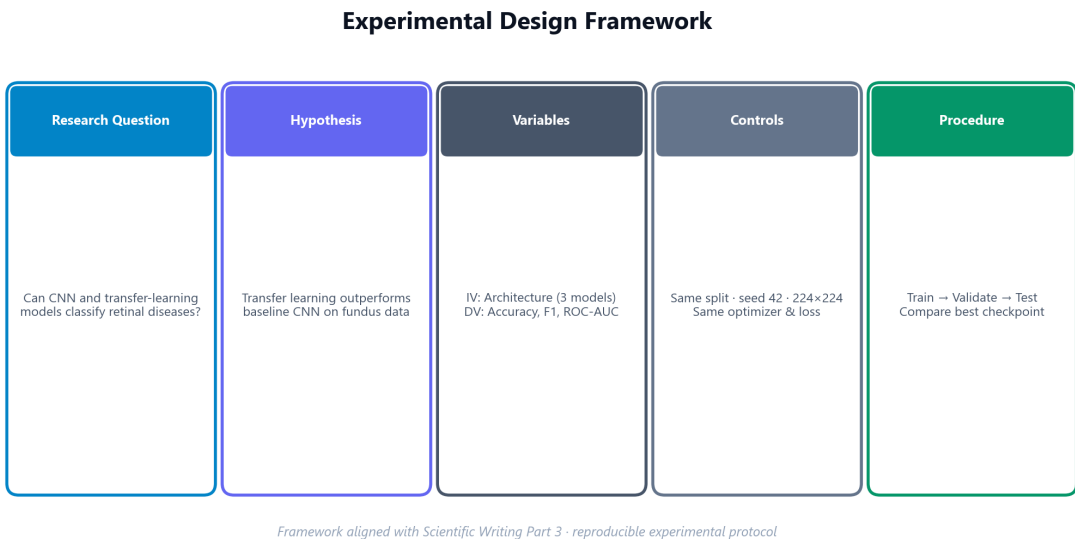
1. Overview

We propose three neural network architectures for four-class retinal disease classification, implemented in src/models.py. We compare a baseline CNN (from scratch) against two transfer-learning models: ResNet50 (residual learning) and EfficientNetB0 (compound scaling).

Model	Type	Key Technique
CNN Baseline	From scratch	Conv + BatchNorm + Dropout
ResNet50	Transfer learning	Residual skip connections
EfficientNetB0	Transfer learning	Compound scaling (depth × width × resolution)

2. Experimental Design Framework

Independent variable: architecture (3 models). Dependent variables: accuracy, F1, ROC-AUC. Controls: same 70/15/15 split, seed 42, 224×224 input, Adam optimizer, categorical cross-entropy.



Framework aligned with Scientific Writing Part 3 · reproducible experimental protocol

Figure 1. Experimental design framework — research question, hypothesis, variables, controls.

3. Proposed Pipeline

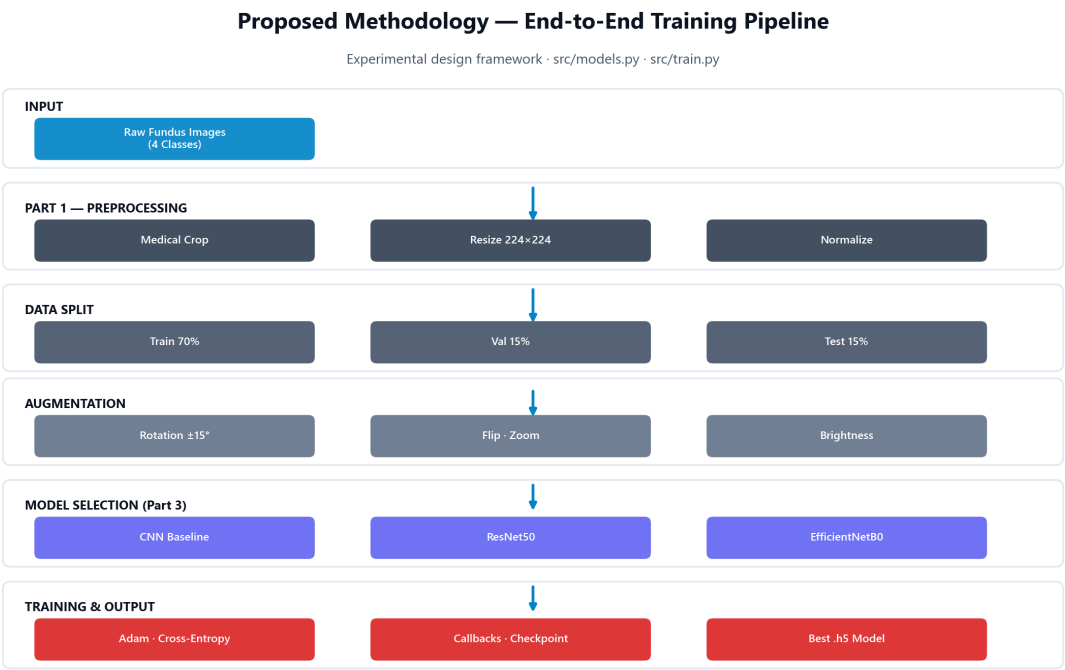


Figure 2. End-to-end training pipeline from raw fundus images to model checkpoint.

4. Architecture 1 — Baseline CNN

Four Conv2D blocks (32→64→128→256 filters) each followed by Batch Normalization, ReLU, and MaxPooling. Global Average Pooling reduces parameters. Two dense layers (512, 256) use BatchNorm, ReLU, and Dropout (0.5). L2 regularization ($\lambda=0.001$) penalizes large weights.



Figure 3. Baseline CNN architecture — Conv blocks, BatchNorm, Dropout, L2 regularization.

Block	Filters	Kernel	After Pool
1	32	3×3	112×112
2	64	3×3	56×56
3	128	3×3	28×28
4	256	3×3	14×14

5. Architecture 2 — ResNet50

ResNet50 uses residual connections $y = F(x) + x$ to enable training of very deep networks. We load ImageNet-pretrained weights, freeze the base, and attach a custom head: GAP → BN → Dropout(0.5) → Dense(512) → Dense(256) → Softmax(4). Optional fine-tuning unfreezes the last 20 layers.

6. Architecture 3 — EfficientNetB0

EfficientNetB0 applies compound scaling — jointly scaling depth, width, and resolution for optimal accuracy-efficiency trade-off. MBConv blocks with Swish activation in the custom head. Dropout rate 0.3 (lighter than ResNet50). ~5M parameters vs ~25M for ResNet50.

Transfer Learning Architectures — ResNet50 vs EfficientNetB0



7. Training Methodology

All models compile with Adam (lr=0.001), categorical cross-entropy, and metrics: accuracy, precision, recall, weighted F1. Callbacks: ModelCheckpoint, EarlyStopping, ReduceLROnPlateau, TensorBoard, CSVLogger.

Parameter	Value
Batch size	32
Epochs	50 (early stopping)
Optimizer	Adam
Learning rate	0.001
Loss	Categorical cross-entropy

7.1 Implementation Reference (src/models.py)

Function	Role
create_baseline_cnn()	Custom 4-block CNN
create_resnet50_model()	ResNet50 + custom head
create_efficientnet_model()	EfficientNetB0 + custom head
compile_model()	Optimizer, loss, metrics
create_model()	Factory: cnn_baseline resnet50 efficientnet
<pre>model = create_model('efficientnet', input_shape=(224,224,3), num_classes=4) model = compile_model(model, learning_rate=0.001, optimizer_name='adam')</pre>	

References

- [1] K. He et al., "Deep residual learning for image recognition," CVPR, 2016.
- [2] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling," ICML, 2019.
- [3] S. Ioffe and C. Szegedy, "Batch normalization," ICML, 2015.
- [4] N. Srivastava et al., "Dropout," JMLR, 2014.
- [5] K. Sainani, Writing in the Sciences Part 3, Stanford Online, 2020.